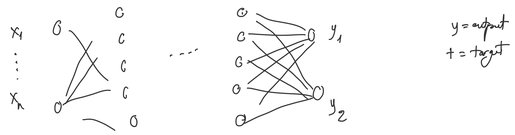
Convolutional Neural Networks

Monday, 26 October 2026 7:41 PM

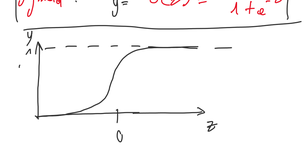
DNN output layers vs tasks



y = output
t = target

1/ Binary Classification

- 2 classes: C_1, C_0 (mutually exclusive)
  - → Only need 1 output
  - Sigmoid: $y = \sigma(z) = \frac{1}{1+e^{-z}}$



- → $P(C_1 | x, w) = y$
  $P(C_0 | x, w) = 1 - y$
- Target: $t = 1 \rightarrow C_1$
  $t = 0 \rightarrow C_0$

2/ Multi-class Classification

- >2 classes
- Single-label vs multi-label
  - Mutually exclusive
  - One class at a time
  - Multiple classes can be active at the same time
- Number of output nodes = number of classes

Target: Single-label

One-hot encoding: $y_c = \begin{cases} 1, & c = c_{target} \\ 0, & \text{otherwise} \end{cases}$



$P(c | x, w) = y_c$ = Softmax$(z) = \frac{e^{z_c}}{\sum_c e^{z_c}}$

make everything positive

Normalization

$\sum y_c = 1$

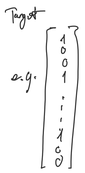
3/ Multi-label Classification



- $P(C | x, w) = y_c$
- Sigmoid: $y_c = \sigma(z) = \frac{1}{1+e^{-z_c}}$
- DO NOT need to add up to 1

Target

e.g.



4/ Regression

- Targets are continuous-valued
- Number of output units = number of variables to be predicted
- $y_n = z_n$ (linear layer)
- Task-specific nonlinearity { sigmoid, ReLU, +Affine transform

Loss functions

1/ Maximum likelihood estimation

w: vector of trainable weights

$L(w | x, t) = P(t | x, w)$ | one training sample (x, t)

$L(w | x_1, \dots x_n) = \prod_n P(t_n | x_n, w)$ | Multiple training sample (x_n, t_n)
$t_1, \dots, t_n$ Independent variables

MLE for Binary Classification

$L(w | x, t) = \begin{cases} y(x, w), & t = 1 \\ 1 - y(x, w), & t = 0 \end{cases}$

$= y(x, w)^t \cdot (1 - y(x, w))^{1-t}$ | one sample

$L(w | x_1 \dots x_n, t_1, \dots, t_n) = \prod_n y(x_n, w)^{t_n} (1 - y(x_n, w))^{1-t_n}$ | Multi sample

Maximizing

→ take -log: $-\sum_n (t_n \log(y(x_n, w)) + (1 - t_n) \log(1 - y(x_n, w)))$

→ Binary cross-entropy

MLE for single-label multiclass Classification

t: index of class

$L(w | x, t) = P(t | x, w) = y_t$ | single samples

$L(w | x_1, \dots x_n, t_1 \dots t_n) = \prod_n y_{t_n}$ | multiple samples

→ Maximize, take -log

$-\sum_n \log(y_{t_n})$

→ Categorical Cross-entropy

MLE for multiple multiclass classification

- Many multiple binary classifications

→ Binary cross-entropy summed over classes and samples

$-\sum_{n,c} (t_{n,c} \log(y_c(x_n, w)) + (1 - t_{n,c}) \log(1 - y_c(x_n, w)))$

MLE for Regression

$t = y(x, w) + n$ (noise $\sim N(0, \sigma^2)$)

If there is infinite additive noise → the $\sum$ follows normal distribution

→ $t \sim N(t | y(x, w), \sigma^2)$

→ $L(w | x, t) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\frac{-(t - y(x, w))^2}{\sigma^2}\right)$

Maximize, take -log

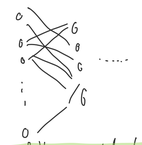
$-\log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) + \frac{(t - y(x, w))^2}{\sigma^2}$

constant; σ^2 constant

$(t - y(x, w))^2$ | one sample

→ Mean squared error | Multiple samples

Convolutional Neural Networks



For MLP, the order of inputs do not matter and each input is treated as independent

→ Bad for inputs whose order matter and they can be dependent
e.g. sound, images, etc.

fully connected layer

Properties

→ Convolution → Translation equivariance

But one kernel is often not enough for pattern detection

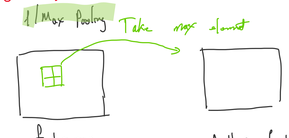
→ Add another kernel and do convolution to another channel

channel: $z_j^c = \sum_k^C x_{j+k} \cdot w_k^c + b^c$

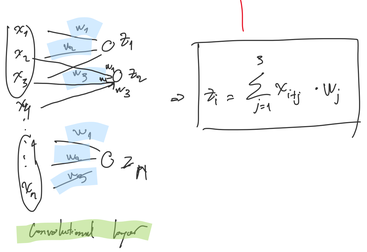
2D CNN layer (images)

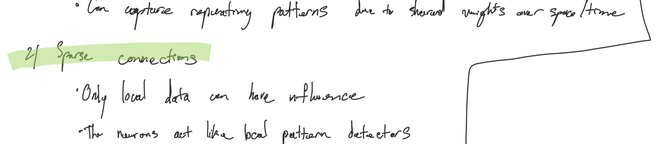
Pooling layers

1/ Max Pooling: Take max element



Makes the feature map locally translation invariant
Often combined with

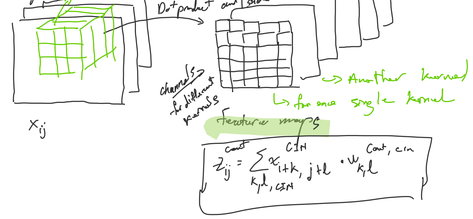


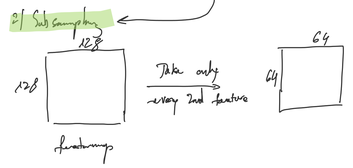


- Can capture repeating patterns due to shared weights over space/time

2/ Sparse connections

- Only local data can have influence
- The neurons act like local pattern detectors

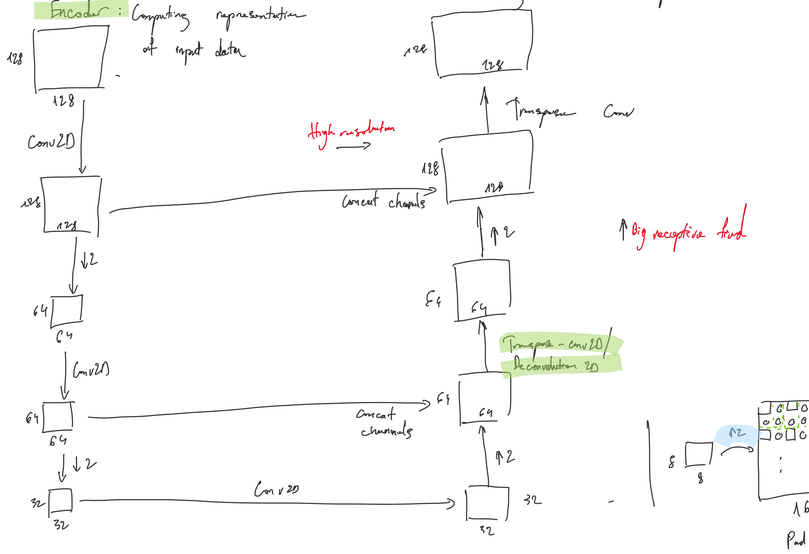


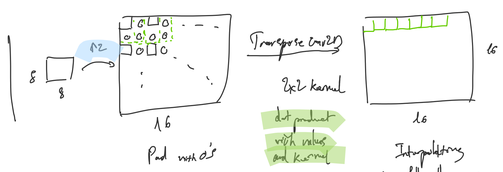


U-nets: encoder – decoder

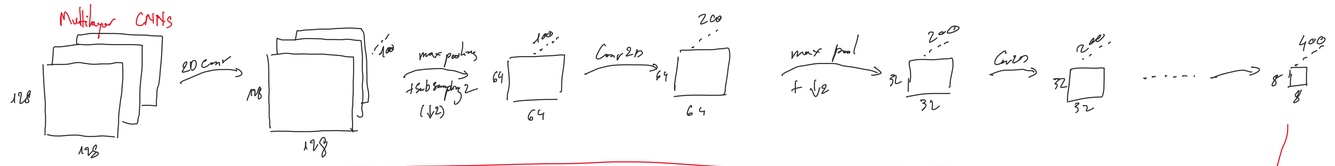
Encoder: Computing representation of input data

Decoding: Decode the representation data

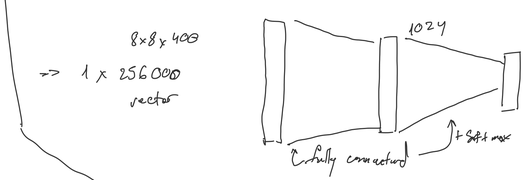




Multilayer CNNs



→ Stack rest as vector

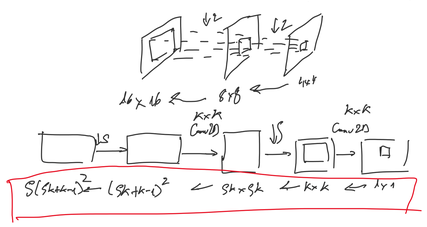


Why do subsampling?

→ Receptive field

The region or pixels of original input that influence feature maps in the deeper layers

More subsampling => More of image reflecting



Examples Choosing which dimension to obtain equivariance

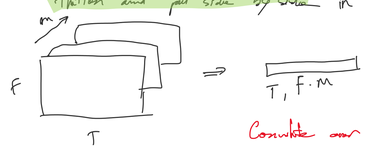
1D → Convolution over time

2D → Convolut over images height width

3D → Convolut over ____ and time

For dimensions that do not need convolution

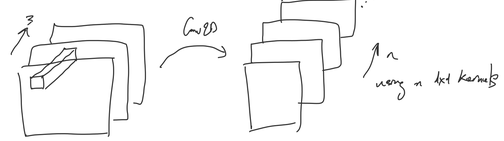
- Flatten and put side by side in dimension we want to convolute over



Convolute over time only

1x1 kernel

→ Why? => Changing dimensionality



3x3 kernel
10 on
20 cont

=> 3x 3x10x20 = 1800 weights

Causal Convolution

- For real-time computing
- Taking past and current results into the convolution

Dilated Convolution

→ Non-Zero weights are exponentially spaced

- Used for capturing of higher spatial spaces for less weights

→ Separate Convolution

Breaking 2Dconv into 2 x 1Dconv

